RELATÓRIO DE CONCLUSÃO

Validar coleta no Prometheus

Projeto: Seminário Técnico — Grafana

Curso: Sistemas de Informação — UniSales

Ambiente validado: Node.js + Prometheus + Grafana + Docker Compose

Status: CONCLUÍDA

Data de validação: 19/10/2026

1. Objetivo da entrega

A tarefa teve como objetivo provar que as métricas expostas pela API são efetivamente coletadas pelo Prometheus e ficam disponíveis para consultas e para consumo pelo Grafana.

2. Configuração validada

No ambiente Docker Compose, o Prometheus utiliza intervalo global de 10 segundos e possui dois jobs: prometheus, para automonitoramento, e api, que consulta o endpoint /metrics da aplicação. Dentro da rede Docker, o alvo da API é api:3000.

```
global:
  scrape_interval: 10s
  evaluation_interval: 10s

scrape_configs:
  - job_name: api
    metrics_path: /metrics
    scrape_interval: 10s
    static_configs:
      - targets: ["api:3000"]
```

3. Validação dos containers e do target

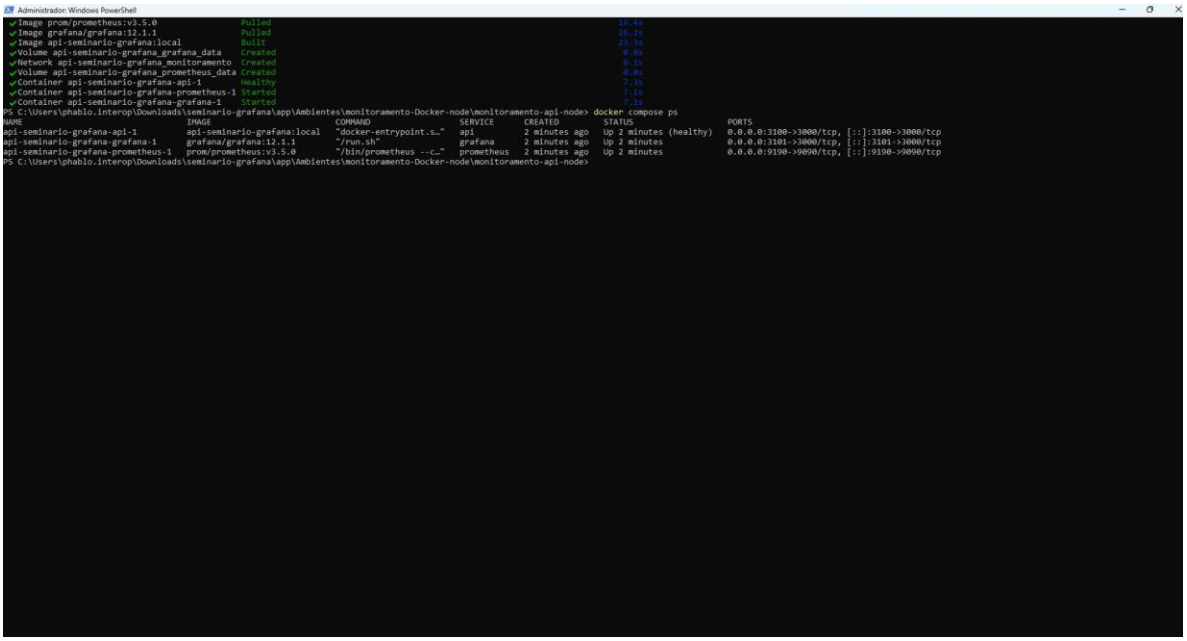


Figura 1 – Serviços api, prometheus e grafana executando pelo Docker Compose.
Fonte: evidência produzida durante a validação prática do projeto (2026).

O serviço da API foi reportado como healthy e os containers de Prometheus e Grafana permaneceram em execução, confirmando que a dependência de inicialização do Compose foi atendida.

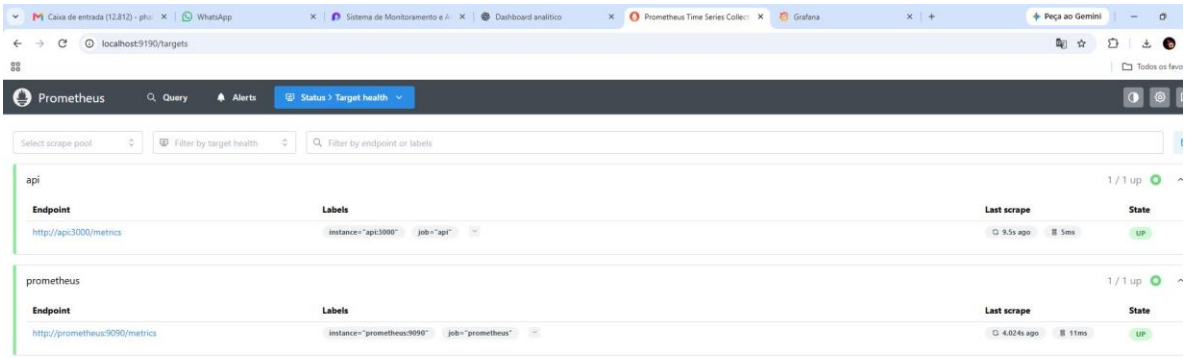


Figura 2 – Página Target health com os jobs api e prometheus em estado UP.
Fonte: evidência produzida durante a validação prática do projeto (2026).

A página de targets mostrou http://api:3000/metrics com estado UP. Isso confirma que o Prometheus consegue resolver o nome do serviço na rede Docker, acessar o endpoint e realizar scrapes com sucesso.

4. Integração com o Grafana

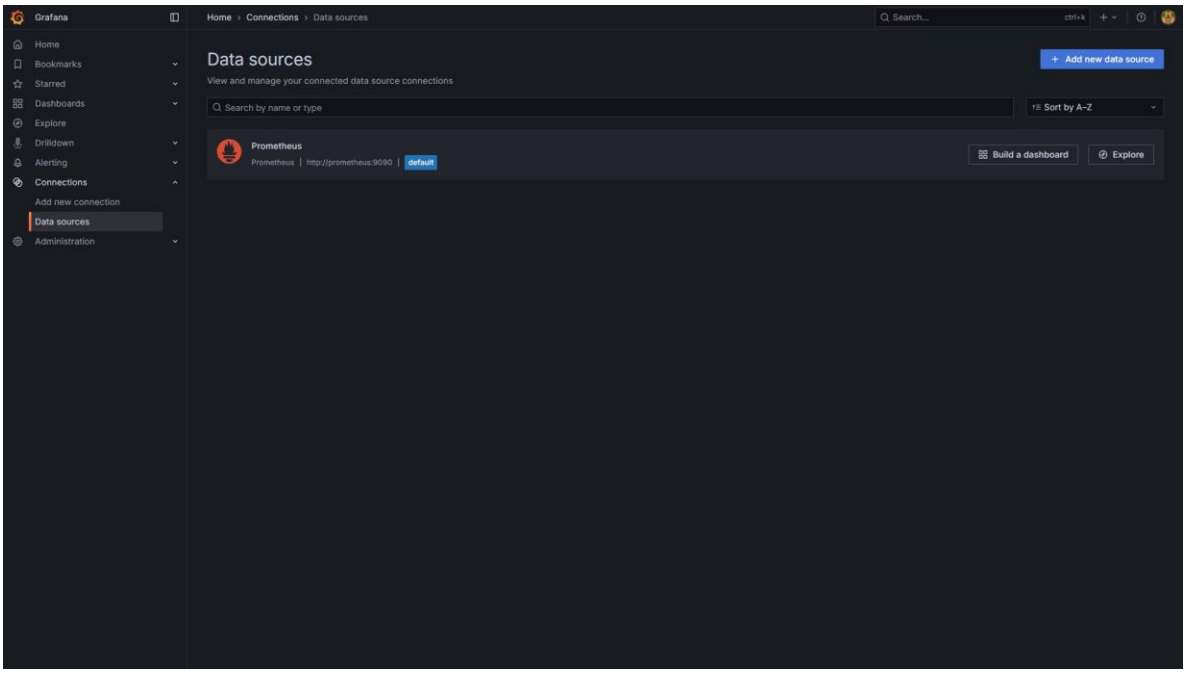


Figura 3 – Data Source Prometheus provisionado automaticamente no Grafana.
Fonte: evidência produzida durante a validação prática do projeto (2026).

O Data Source foi criado por provisioning e utiliza http://prometheus:9090 como URL interna. Isso demonstra que as métricas coletadas pelo Prometheus estão disponíveis para a camada de visualização sem configuração manual após o start do Compose.

5. Alteração em relação ao planejamento original

Planejamento original	Implementação validada
A validação inicial havia sido planejada em execução local, com Prometheus acessando localhost:3000.	A validação definitiva foi feita no ambiente Docker Compose. O Prometheus acessa api:3000 e o Grafana acessa prometheus:9090 pela rede interna do Compose.

Impacto da mudança: A mudança tornou o teste mais representativo da arquitetura final e eliminou dependências de portas internas entre os serviços.

6. Critérios de conclusão

Critério	Resultado	Status
Endpoint de métricas	/metrics acessível pela rede Docker.	**Concluído**
Target da API	api:3000 em estado UP.	**Concluído**
Scrape periódico	Intervalo de 10 segundos configurado.	**Concluído**
Automonitoramento	Job prometheus também em estado UP.	**Concluído**
Consumo pelo Grafana	Data Source provisionado e funcional.	**Concluído**

7. Conclusão

A coleta foi validada de ponta a ponta no ambiente final de containers. O Prometheus recebe as métricas da API, mantém seu próprio job de monitoramento e fornece os dados ao Grafana pela rede Docker.